

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I M V Veeranjaneya Reddy(192472140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M V Veeranjaneya Reddy

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Edge Detection in Low Contrast Images

An image has low contrast, causing edge detection algorithms to fail.

Analyze the problem and explain how preprocessing enhances edge detection results.

Analysis:

- Edges are detected based on changes in pixel intensity.
- In a low-contrast image, these changes are very small.
- Noise may become comparable to the actual edge information.
- As a result, edges may be weak, broken, or completely missed.

Preprocessing methods:

1. **Contrast stretching** – increases the difference between dark and bright regions.
2. **Histogram equalization** – improves the overall contrast of the image.
3. **CLAHE** – improves local contrast while avoiding excessive enhancement.
4. **Gaussian filtering** – reduces noise before edge detection.
5. **Normalization** – scales pixel intensities to a suitable range.

2. Corner Detection in Textured Regions

A system applies corner detection in highly textured regions, resulting in excessive keypoints.

Analyze the issue and suggest methods to improve feature selection.

Analysis:

- Texture produces many corners and local intensity variations.
- Excessive keypoints increase computation time.
- Some detected points may be unstable or redundant.
- Too many similar features can reduce matching efficiency.

Methods to improve feature selection:

1. **Response thresholding** – keep only corners with strong detector responses.
2. **Non-Maximum Suppression (NMS)** – remove nearby weaker keypoints.
3. **Minimum distance constraint** – maintain a minimum distance between selected points.
4. **Adaptive thresholding** – use different thresholds depending on local texture.
5. **Feature ranking** – select the strongest and most distinctive keypoints.

3. Feature Matching with Noise

Feature matching between two images fails due to noise in one image.

Analyze the impact of noise and explain how preprocessing improves matching accuracy.

Impact of noise:

- Creates false edges and keypoints.
- Changes local image patterns.
- Makes descriptors less reliable.
- Produces incorrect feature correspondences.
- Reduces matching accuracy.

Preprocessing methods:

1. **Gaussian filter** – removes Gaussian noise.
2. **Median filter** – effective for salt-and-pepper noise.
3. **Bilateral filter** – reduces noise while preserving edges.
4. **Image normalization** – makes intensity distributions more consistent.
5. **Contrast enhancement** – improves useful feature information.

Matching improvement:

- Apply noise removal before feature detection.

- Use robust descriptors such as **SIFT** or **ORB**.
- Use **Lowe's ratio test** to reject ambiguous matches.
- Use **RANSAC** to remove incorrect feature correspondences.

4. PCA and Information Loss

A system applies PCA aggressively and loses important feature details.

Analyze the trade-off between dimensionality reduction and information loss, and justify optimal component selection.

Trade-off:

More Components	Fewer Components
More information retained	More information lost
Higher dimensionality	Lower dimensionality
More computation	Faster computation
Less reconstruction error	Higher reconstruction error

Optimal component selection:

1. Calculate the covariance matrix.
2. Find eigenvalues and eigenvectors.
3. Sort components according to eigenvalues.
4. Calculate **explained variance ratio**.
5. Select enough components to retain approximately **90–95% of the variance**, depending on the application.
6. Validate the selected number using classification or matching performance.

5. Combined Feature Detection and Matching

A system detects features correctly but fails during matching due to incorrect descriptor selection.

Analyze the issue and explain how appropriate feature descriptors improve matching performance.

Analysis:

- A detector finds important points in an image.
- A descriptor represents the local region around each point.
- If the descriptor is unsuitable, correct keypoints may still produce poor matches.
- Different descriptors have different strengths and limitations.

Suitable descriptors:

Descriptor Main Advantage

SIFT	Robust to scale and rotation changes
SURF	Fast and robust to scale changes
ORB	Fast and suitable for real-time applications
BRIEF	Simple and computationally efficient
HOG	Useful for shape and object representation

Example:

If the image contains scale and rotation changes, **SIFT** is a good choice. For real-time applications where speed is important, **ORB** is more suitable.

Matching process:

1. Detect keypoints.
2. Extract appropriate descriptors.
3. Compare descriptors using a suitable distance measure.

4. Apply ratio test or cross-checking.
5. Use **RANSAC** to remove incorrect matches.